

Document number: P0773R0

Date: 2017-10-05

Project: ISO JTC1/SC22/WG21, Programming Language C++

Audience: Library Evolution Working Group

Reply to: Arthur O'Dwyer <arthur.j.odwyer@gmail.com>, Bob Steagall <bob.steagall.cpp@gmail.com>

Towards meaningful fancy pointers

[Summary. Intended reading order.](#)

[Objects and values. A handle is a value that points to an object. Casting.](#)

[Storage representation.](#)

[An allocator is a handle to a memory resource.](#)

[Pointer values may be dereferenceable and/or deallocatable.](#)

[Purposes of fancy pointer types.](#)

[Scenario A. "Offset" pointers with a modified bit-level representation.](#)

[Scenario B. Small-data-area "near" pointers.](#)

[Scenario C. High-memory-area "far" pointers.](#)

[Scenario D. "Fat" pointers carrying metadata for dereference-time.](#)

[Scenario E. "Segmented" pointers carrying metadata for deallocate-time.](#)

[Is any kind of fancy pointer compatible with C++?](#)

[Requirements on fancy pointer types if *fancy-pointer* support were adopted. *basic_string* requires trivial default constructibility.](#)

[Appendix. Representation-awareness of container and iterator types, in some present-day library implementations.](#)

[Appendix. References and further reading.](#)

[Acknowledgments.](#)

Summary. Intended reading order.

This is a position paper explaining my take on fancy pointers in C++. Indented, grayed paragraphs represent "clarifications" that should be skipped on your first reading.

For example, this grayed paragraph should be skipped on your first reading.

I contend that the Standard's current wording related to fancy pointer types is so vague and unconstraining as to be effectively meaningless. For example, the table in [allocator.requirements] requires that the fancy pointer `p` passed to `deallocate` "shall be a value returned by an earlier call to `allocate`...", but the Standard lacks any formal notion of what it means to preserve the "value" of a fancy pointer. (In this paper's Scenarios C, D, and E, casting the allocated fancy pointer to *native pointer* type and then back to fancy pointer type is not a value-preserving operation.) Some operations on allocators seem to have implicit requirements; for example, `allocate_shared` implicitly requires that fancy pointers be convertible to native pointers. Some vendors' container implementations require things of fancy pointer types beyond the Standard; for example, `basic_string` usually requires that its pointer type be trivially constructible.

This paper tries to start a discussion of formal semantics for "fancy pointer" types in C++, and what it means for a container or algorithm to be "allocator-aware."

I propose a taxonomy of fancy pointer types into five not-mutually-exclusive categories, along the axes of *storage representation*, *addressable range*, and *carried metadata*.

- (A) Fancy pointers with a different bit-level representation than native pointers: **offset** pointers. This is explicitly supported today. Many types (`std::vector`) work already; some non-container types (`std::shared_ptr`) do not work.
- (B) Fancy pointers addressing a narrower range of memory (a Small Data Area): **near** pointers. This is plausible. Some containers (`std::vector`) implicitly work already; some (`std::list`) work only in certain runtime situations.
- (C) Fancy pointers addressing a wider range of memory: **far** pointers. This turns out to be impossible in C++ because every object must have a native address.
- (D) Augmented fancy pointers whose metadata affects arithmetic: **fat** pointers. This is plausible but expensive, and not worth the implementation effort for vendors.

- (E) Augmented fancy pointers whose metadata affects only deallocation: **segmented** pointers. This is plausible. Some containers (`std::vector`) work already; some (`std::list`) require only trivial modifications; some (`std::shared_ptr`) require major fixes.

I propose two self-consistent future directions for fancy pointers. In one direction, we keep the status quo that no scenario except (A) is supported, and perhaps move toward separation of the *pointer storage type* from the *allocation mechanism*. In my preferred direction, we require that vendors support scenarios (B) and (E) as well as (A).

In the sections that follow, I will lay out the case for these recommendations.

Objects and values. A handle is a value that points to an object. Casting.

By **object** I mean something like we see in object-oriented programming: an *object* has a unique identity, and this identity is preserved even as the internal state of the object is mutated. Objects with different identities are never interchangeable. In C++, an object is often represented in code by a C++ variable or piece of memory, in which case the object's identity is tantamount to that variable's memory address. However, there also exist objects that are not variables. For example, "the program call stack" could be considered an object; it is mutated whenever a function is called or returns. "The heap" is another object; it is mutated by operator `new` and operator `delete`.

By **value** I mean something like we see in "value semantics": a *value* is an almost Platonic ideal, such as 5 or "hello". Values can exist independently of any particular storage. Identical values are interchangeable; saying that value a is equal to value b is tantamount to saying that value a *is* value b.

Sometimes we say that an object "has a value," or refer to an object's state as "its value." I will try not to do either of those things in this paper.

By **handle** I mean a value representing the identity of an *object*. You cannot have two different objects with the same identity, but you can have two copies of a handle — two handles — that refer to the same object. Handles that refer to the same object are "equal," and thus (philosophically speaking) interchangeable.

In C++, a *handle* is often represented in code by (the value of) a C++ native pointer. A handle does nothing but identify an object; therefore a handle can be copied; therefore a handle does not "own" its referent.

A **type** is essentially a set of values. In C++ we usually say that a value "has a type," but in this case I'm being more Platonic. Suppose I make a type `digit` which is the set of values 0 through 9; and I make another type `evendigit` which is the set of values 0,2,4,6,8. The value 4 belongs to both types; the value 4 is representable in both types.

This definition of *type* is problematic in C++ because we are used to strong static typing, where a value really *does* "have a type." We want to say that the value 4 is representable in both `digit` and `evendigit` — that `digit(4)` and `evendigit(4)` represent "the same value," even though they have different C++ types and may not even be comparable via `operator==`. Therefore, C++ introduces the idea of *casting*.

A value of type A may be **cast to** type B. The result is a value of type B which nevertheless is in some abstract sense *the same* as the value of type A. Value-sameness is preserved through casting operations, when possible. If the source value is not representable in the destination type, then value-sameness cannot be preserved; such a cast might be prohibited by the language, or it might just throw away some parts of the value.

For example, casting the value 3.00 from `double` to `int` and back will preserve its value, because the value "3" is representable in both types. Casting the value 3.14 from `double` to `int` and back will preserve part of its value but throw away some of it. In C++, "casting" of the kind we're talking about is usually represented via `static_cast`.

If a type T is the Cartesian product of types U and V, *and* there is a natural mapping from T to U, we say that T is an **augmented type** with respect to U. *Casting* an augmented type T to its corresponding non-augmented type U throws away the "V part" of the T value but preserves the "U part." Casting from U to T may be possible, but will fail to correctly reconstitute the "V part" of the data. I will use the term **metadata** to refer to the "V part" of the data, the part that is thrown away when casting to the non-augmented type.

For example, `double` is more or less an augmented type with respect to `int`. Casting the value 3.14 from `double` to `int` will preserve the `int` part of its value but throw away the fractional part. (When considering `double` as an augmented `int`, the

fractional part is the *metadata*.) Casting from `int` back to the augmented type `double` is possible, but will reconstitute the metadata as `".00"` instead of the correct `".14"`.

Most well-known product types, such as `std::tuple<U, V>`, are not really augmented types because there is no "natural" mapping from `std::tuple<U, V>` to `U`. In C++ terms, `static_cast`'ing from `std::tuple<U, V>` to `U` will not compile. But the following `Augmented<U, Meta>` is definitely an *augmented type* with respect to `U`:

```
template<class U, class Meta>
struct Augmented {
    U u_;
    Meta meta_;
    Augmented(U u, Meta m) : u_(u), meta_(m) {}
    operator U() const { return u_; }
};
```

In C++, a derived type is often an augmented type with respect to its public base type(s). Casting from derived to base results in *slicing* away of the "non-base parts" of the instance, but preserves the "base parts." If casting from derived to base is forbidden (for example by deleting the base type's copy constructor), then the derived type is *not* an augmented type with respect to the base type.

Storage representation.

In C++, a type defines not only a set of *values* but also a mapping from values to bit-level *storage representations*. (For example, while I consider `(double)3` and `(int)3` to be the same *value*, they have different bit-level *storage representations* when they are stored into memory as part of a C++ object.)

In some programming systems the most important thing about a C++ type is not its range of possible values but its mapping from values to *storage representations*.

An example of significant *storage representation* is `boost::interprocess::offset_ptr<T>`, which has the same range of values as the native pointer type `T*` but a different bit-level storage representation. Since `(T*)(&x)` and `offset_ptr<T>(&x)` are the same *value*, we should expect that they can be interconverted using `static_cast`. In fact, under Boost 1.64, `static_cast<offset_ptr<T>>((T*){})` is accepted but `static_cast<int*>(offset_ptr<T>{})` is ill-formed. I believe this is a defect in the current implementation of `boost::interprocess::offset_ptr`.

An allocator is a handle to a memory resource.

A *memory resource* is an object that affords two functions: *allocation* and *deallocation* of memory chunks. How it does that is irrelevant for our purposes. An allocation mechanism must somehow yield a *handle* that uniquely identifies the allocated memory chunk; the corresponding deallocation mechanism must accept that same *handle* as input.

Instances of types derived from `std::pmr::memory_resource` are memory resources. Instances of `boost::interprocess::segment_manager_base` are also memory resources. But another well-known memory resource is just "the heap." "The heap" doesn't exist as a concrete C++ object, but it still affords *allocation* via operator `new` and *deallocation* via operator `delete`. Therefore I say that "the heap" is a *memory resource* as well.

An *allocator* is a handle to a memory resource. By this definition, an instance of `std::allocator<T>` is an allocator, because it is a handle to "the heap." An instance of `std::pmr::memory_resource*` also is an allocator, because it is a handle to a `std::pmr::memory_resource`. (Recall that a *handle* is a value that uniquely identifies an object.)

In order to enable generic programming with allocators, the STL introduced the `Allocator` concept. This is a standardized interface that must be provided by any type that is going to be used as the allocator of an STL container. A type that models `Allocator` must provide actual *member functions* named `allocate` and `deallocate`. For example, the type `std::pmr::memory_resource*` does not model `Allocator`, but the type `std::pmr::polymorphic_allocator<T>` does model `Allocator`.

In C++ there is actually a whole family of concepts `Allocator<T>` each associated with allocating and deallocating C++ objects of type `T`. However, it is implicitly required that I can *cast* an object of type `some_allocator<T>` to type `some_allocator<U>`. Casting alters type without altering value; that is, casting a handle in this way produces a new handle which uniquely identifies the same memory resource.

Determining the appropriate destination type for a cast is called **rebinding**. C++ implicitly requires that if x (of type X modeling `Allocator<T>`) is a handle to some memory resource m , then after the variable definition `auto y = static_cast<std::allocator_traits<X>::rebind_alloc<U>>(x)`, y (of type Y modeling `Allocator<U>`) is a handle to that same memory resource m .

This definition of rebinding is compatible with "slab allocator"-style memory resources, where `SlabAllocator<int32_t>::allocate` and `SlabAllocator<int64_t>::allocate` allocate chunks out of different, non-overlapping slabs of memory. A single *memory resource* then consists of a single *complete set* of slabs. The important thing is that we must be able to take a handle to this *memory resource* and cast it around without affecting its *value* (that is, the identity of the memory resource to which it refers).

```
SlabResource mr1;
auto a32 = SlabAllocator<int32_t>(&mr1); // get a handle to the memory resource
auto b64 = SlabAllocator<int64_t>(a32); // cast that handle to a new C++ type
auto c32 = SlabAllocator<int32_t>(b64); // cast the handle back to the original type
// At this point, a32 and c32 must both identify memory resource "mr1".
// c32 is NOT ALLOWED to identify a different memory resource "mr2".
// c32 is NOT ALLOWED to identify no memory resource (e.g. to have become "null" or otherwise unusable).
// a32 and c32, being the same value in the same C++ type, must be truly interchangeable.
```

The foregoing definitions imply that allocators are cheap to copy. Making a copy of an allocator does not make a copy of the underlying memory resource. Allocators are rebindable and castable: you can change the C++ type of an allocator instance without changing its fundamental value. These rules should be familiar because this is also how C++ treats *iterators* and *pointers*: handles, cheap to copy, castable.

We can create an allocator *type* which has many possible *values*: for example, `std::pmr::polymorphic_allocator`. We can also create an allocator type whose set of possible values has only one member: for example, `std::allocator` is an allocator type whose only possible value is "the identity of 'the heap'." Since the set of possible values has size 1, an instance of the type contains $\log(1) = 0$ bits of information and thus `std::is_empty_v<std::allocator<T>>`. Casting an `std::allocator<T>` to `std::allocator<U>` preserves its value, in that the value resulting from the cast continues to identify "the heap."

Pointer values may be dereferenceable and/or deallocatable.

A **native pointer** is a value of a native pointer type. A **native pointer type** is a C++ type of the form τ^* where τ is an object type. For the purposes of this paper, we don't care about function pointers or member pointers, because those aren't suitable subjects for allocation and deallocation.

A **pointer** is a handle that satisfies specific syntactic constraints; it must afford (many of) the same operations as a C++ native pointer. Specifically, the following operations must be supported:

<code>decltype(p){}</code>	(value-initialization must create a null value)
<code>p == nullptr, if (p), !p</code>	(comparison against null)
<code>p == p, p != p</code>	(comparison against another pointer value of the same type)
<code>*p</code>	(dereference)

Native pointers also support the **pointer arithmetic** operations, which is to say, native pointers model the `RandomAccessIterator` concept:

<code>++p, p++, --p, p--</code>	(bidirectional mutation)
<code>p += i, p -= i</code>	(random-access mutation)
<code>p + i, i + p, p - i</code>	(random-access construction)
<code>p - p</code>	(random-access difference)
<code>p < p, p <= p, p > p, p >= p</code>	(random-access inequality comparison)
<code>p[i]</code>	(random-access dereference)

A **fancy pointer** is a pointer that is not a native pointer. A **fancy pointer type** is a C++ type all of whose values are fancy pointers. When I need to refer to an example C++ fancy pointer type, I will use names such as `FancyPtrInt` (for a pointer whose values identify objects of type `int`) or `FancyPtrT` (for a pointer whose values identify objects of type τ).

Fancy pointer types are described in [allocator.requirements]/5 as types modeling both `NullablePointer` and `RandomAccessIterator`; that is, in today's C++, a fancy pointer type must provide pointer arithmetic. I will argue that fancy pointer types have no philosophical need to provide pointer arithmetic.

Some pointer values are *dereferenceable* and some are not.

Null is a specific pointer value. Every pointer type contains the value *null*. Casting the null value preserves its value. The null value is never dereferenceable.

In the native pointer type `int*`, we can construct non-dereferenceable values in at least the following ways:

```
int arr[10];
int *p1 = nullptr; // the null pointer is never dereferenceable
int *p2 = arr + 10; // a "one-past-the-end" pointer is not dereferenceable
```

There are also ways to have an object `p` of type `int*` where the behavior of `*p` is undefined, such as

```
int *p3 = new int; delete p3; // a deallocated pointer is not dereferenceable
int *p4; // an uninitialized pointer is not dereferenceable
```

but these examples are not interesting in the context of this paper. Philosophically it is arguable that these examples treat `p3` and `p4` as *stateful objects*, and that the names `p3` and `p4` do not identify pointer *values* at all. In this paper I am concerned with pointer values like `p1` and `p2`; I do not consider `p3` and `p4` to be relevant.

Some pointer values are *deallocatable* (with respect to a given memory resource) and some are not. Any pointer value successfully returned from the *allocation* mechanism of a memory resource `mr` is deallocatable with respect to `mr`.

Many *dereferenceable* pointers will not be *deallocatable*. (For example, any pointer identifying a non-allocated object will not be deallocatable by any allocator. Also, a pointer allocated by memory resource A is unlikely to be deallocatable by memory resource B.) Some *deallocatable* pointers will not be *dereferenceable*. (For example, the native pointer returned from `malloc(0)` on some implementations is deallocatable with respect to `malloc()/free()` but not dereferenceable.)

Purposes of fancy pointer types.

What are the possible motivations for fancy pointer types? Why might a programmer desire a pointer type that is different from any native pointer type? Here are five scenarios I consider plausible.

Scenario A. "Offset" pointers with a modified bit-level representation.

Some programming systems use *offset pointers*, which are equivalent to native pointers but whose bit-level storage representation is different. For example, the storage representation of a `boost::interprocess::offset_ptr` identifying an object `x` is the memory address of `x` minus the address of the `offset_ptr` itself.

This means that the storage representation of a `boost::interprocess::offset_ptr` object depends on the object's own memory address. `offset_ptr` objects are not trivially copyable. Yet, at the Platonic level, `offset_ptr` can be said to have the same set of possible *values* as a native pointer type.

The benefit of `boost::interprocess::offset_ptr` is that if a memory region is mapped into the address space of two different processes, then every `offset_ptr` residing in the mapped region will identify the same object no matter which process is asking — as long as the identified object *also* resides in the mapped region.

Can we use *offset fancy pointer types* to enable safe sharing of memory regions?

Yes; but the requirements on vendors are unclear, and there are pitfalls for the programmer.

A C++ object type is *representation-aware* if its object representation stores handles to allocated memory only in objects of the appropriate fancy pointer type. Instances of *representation-aware* types instantiated with `boost::interprocess::offset_ptr` can safely be stored in shared memory segments; instances of *non-representation-aware* types cannot safely be stored in shared memory segments.

The following standard types are required or expected to be *representation-aware*:

- Allocator-aware container types. (But `libstdc++`'s containers break this rule.)
- Container "node handle" types.

- `basic_string`. The usual "small string optimization" does not affect *representation-awareness*. In both `libc++` and `libstdc++`, the relevant type-pun involves the representation of the `size_t` `capacity` member, not the representation of the allocated data pointer.
- `unique_ptr`.

The following standard types are, in practice, *not* representation-aware:

- Container "iterator" types. (See [Representation-awareness of iterators](#).)
- `shared_ptr`. But `boost::interprocess::shared_ptr` is. An instance of `boost::interprocess::shared_ptr<T,A,D>` holds fancy pointers (of type rebound from `allocator_traits<A>::pointer`) to the controlled object and to the control block.
- `packaged_task`, `promise` and `future`.
- `function` and `any`.
- `lock_guard`, `unique_lock`, `string_view`, etc.
- `pmr::polymorphic_allocator`. But `boost::interprocess::allocator` is. An instance of `boost::interprocess::allocator<T,MR>` holds a fancy pointer (of type rebound from `MR::void_pointer`) to the underlying memory resource of type `MR` (typically `boost::interprocess::segment_manager`).
- Any polymorphic class type — for example `pmr::memory_resource`, `ostream`, etc. — because of the `vptr`.

The *representation-awareness* of a C++ class type is orthogonal to its memory allocation strategy.

For example, `shared_ptr` requires allocation (and supports allocation with an allocator via `allocate_shared`) but `shared_ptr` is not representation-aware. Vice versa, `boost::interprocess::allocator` is representation-aware (it contains a non-static data member of fancy pointer type) but not allocator-aware (the fancy pointer is not obtained via `allocator_traits<A>::pointer` for any type `A`, and does not point to an allocation).

It is a shame that the standard *representation-aware* types inherit their storage type from `allocator_traits<A>::pointer` instead of from a separate template parameter `P`. I don't see how to fix this without breaking ABI and requiring a lot of work from vendors. A small step in this direction would be to add an allocator adaptor `std::storage_allocator_adaptor<P,A>` identical to `A` except that its pointer typedef is `pointer_traits<P>::rebind<A::value_type>`. This would ease using `offset_ptr` with representation-aware containers; but not help with types that were not already representation-aware, such as `shared_ptr`.

The programmer is responsible for discovering whether any given C++ type is *representation-aware*. There is no programmatic way to determine the *representation-awareness* of a given type. The consequence of wrongly guessing that a type is *representation-aware* is typically a segfault.

There are three alternatives applicable to the current situation:

1. Offset fancy pointer types are allowed, but difficult to use safely. (The current solution.)
2. Offset fancy pointer types are not allowed. The Standard removes support for `boost::interprocess::offset_ptr`. This solution breaks existing code.
3. Offset fancy pointer types become more fully supported. Many additional standard types gain *representation-awareness*; for example, `std::shared_ptr` gains a template parameter controlling the type of its internal pointers. This solution requires vendors to do a lot of new work, and breaks ABI compatibility. This solution does not address the fundamental problem that there is no programmatic way to determine or ensure the *representation-awareness* of a given type.
4. The pointer type associated with the *object representation* of a class is somehow divorced from the pointer type associated with the allocation mechanism. Each allocator-aware type gains a template parameter `P` controlling the type of its internal pointers, in addition to the old template parameter `A` controlling its allocation mechanism. This solution requires vendors to do a lot of new work, and breaks ABI compatibility. This solution again does not address the fundamental problem that there is no programmatic way to determine or ensure the *representation-awareness* of a given type.

In my opinion, solution (1), the de-facto solution today, is the only tenable solution. Solution (4) is philosophically attractive but does not make any headway on the fundamental problem and therefore offers no benefit at present. We are left with solution (1): **Offset fancy pointer types are allowed, but difficult to use safely.**

Scenario B. Small-data-area "near" pointers.

Some programmers deal in memory "arenas" that are much smaller than the entire range of main memory. The programmer may wish to use a C++ pointer type that reflects that smaller range of possible pointer values. Since this pointer type contains fewer than (address-space) values, its representation in memory requires fewer than $\log(\text{address-space})$ bits. For example, a pointer type that can hold one of

only 32,767 distinct addresses (or *null*) philosophically ought to require only $\log(32768) = 15$ bits of storage per pointer. A C++ fancy pointer type with fewer value bits than a native pointer is called a *near fancy pointer*.

On a 32-bit microcontroller, the programmer might define a frequently used list container like this:

```
#include <list>
#include <memory_resource>

// Reserve representation 0x0000 for "null".
std::pmr::monotonic_buffer_resource g_smallData((void*)0x0001, 32767, std::pmr::null_memory_resource());

int main() {
    std::pmr::list<int32_t> widgets(&g_smallData);

    widgets.push_back(1);
}
```

Any allocations made by `widgets` always reside within the "small data area," addresses 0x00000001 through 0x00007FFF. This means that in principle, each pointer should consume only 16 bits of memory footprint. Since each list node contains two pointers, this would be a savings of 4 bytes per node, allowing us to fit twice as many list elements into the small data area as we could naively fit.

The Standard Library doesn't provide a fancy-pointer version of `memory_resource`, but we can easily roll our own. We'd create a near fancy pointer type like this:

```
template<class T>
class NearPtr {
    int16_t ptr_ = 0;
public:
    NearPtr() = default;
    NearPtr(std::nullptr_t) {}
    explicit NearPtr(T *p) : ptr_((int16_t)(intptr_t)p) {}
    T& operator*() const {
        return *(T*)(intptr_t)ptr_;
    }

    template<class U>
    explicit NearPtr(const NearPtr<U>& rhs) : ptr_(rhs.ptr_) {}
};

// Reserve representation 0x0000 for "null".
std::pmr::monotonic_buffer_resource g_smallData((void*)0x0001, 32767, std::pmr::null_memory_resource());

template<class T>
struct NearAllocator {
    using value_type = T;
    using pointer = NearPtr<T>;
    using void_pointer = NearPtr<void>;

    pointer allocate(size_t n) {
        void *vp = g_smallData.allocate(n * sizeof(T), alignof(T));
        return static_cast<pointer>(static_cast<void_pointer>(vp));
    }
    void deallocate(pointer p, size_t n) {
        g_smallData.deallocate(static_cast<void*>(static_cast<T*>(p)), n * sizeof(T), alignof(T));
    }
};

int main() {
    std::list<int32_t, NearAllocator<int32_t>> widgets;

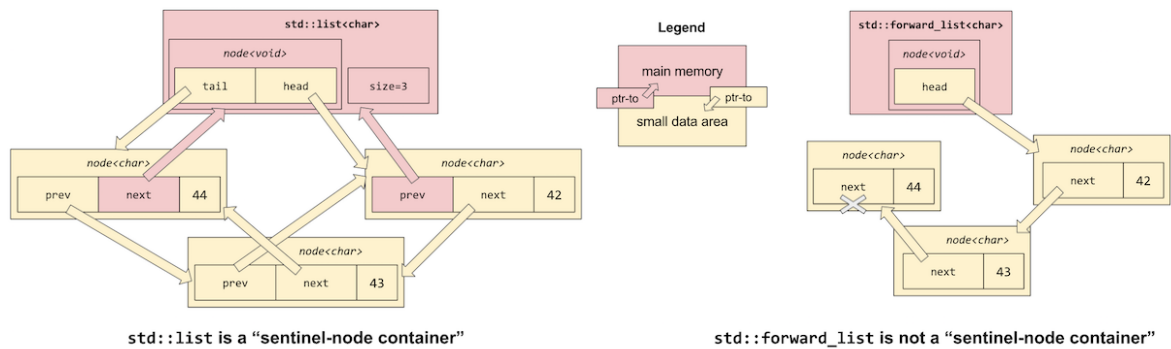
    widgets.push_back(1);
}
```

Can we use *near fancy pointer* types to address objects that are located in a "small" fixed subset of main memory?

Yes; but there are practical limitations.

Near fancy pointer types (such as the `NearPtr` above) are no problem for these standard containers: `deque`, `forward_list`, `vector`. But these other standard containers *will* have technical difficulties with near fancy pointers: `list`, `map`, `set`, `unordered_set`, `unordered_map`. These containers have trouble because their natural implementation requires a linked list where most nodes in the linked list are allocated (i.e. fancy pointers identifying objects in the "small data area") and some "sentinel node" is not (i.e. the sentinel node is not located in the "small data area" and thus is not addressable by the near fancy pointer type).

Containers with this difficulty shall be called *sentinel-node containers*.



An implementation of `std::list` where the sentinel node is allocated (rather than being part of the container instance's member data) is not a *sentinel-node container*.

We have the following possible solutions to this problem:

1. Near fancy pointer types are just not allowed. (This is the reference solution. Any solution less appealing than this one can be summarily dismissed.)
2. Near fancy pointer types are not allowed for *sentinel-node containers*, but are allowed for other containers. The Standard would have to specify for each container whether it was list-like or not.
3. Sentinel-node containers use only native pointers for addressing. These native pointers are cast back to fancy pointers at deallocation time. This solution loses the memory-footprint benefit we were hoping to get from using fancy pointers. This solution is incompatible with Scenarios A and E. (Therefore this solution is summarily dismissed.)
4. Each node of a sentinel-node container holds an extra bit indicating whether the node is allocated (in which case it is identified by a fancy pointer) or non-allocated (in which case it is identified by a native pointer). This solution loses the memory-footprint benefit we were hoping to get from using fancy pointers. This solution is incompatible with Scenario A. (Therefore this solution is summarily dismissed.)
5. Near fancy pointer types are allowed for sentinel-node containers, but if the container cannot statically detect that its pointer type is wide enough to access all of main memory, then the container will switch to an implementation where the sentinel node is allocated using the same mechanism as other nodes. This solution essentially requires that vendors provide two implementations of each sentinel-node container: one for efficiency and one for use with near fancy pointers.
6. Near fancy pointer types are allowed for sentinel-node containers, but if the container itself, or any subobject thereof, cannot be addressed with (a rebinding of) its own pointer type, then the behavior is undefined. This solution does not require any new work from vendors, but it does introduce a new source of undefined behavior.

```
int main() {
    using L = std::list<int32_t, NearAllocator<int32_t>>;

    L alpha = std::make_shared<L>(); // undefined behavior
    auto beta = std::make_shared<L>(); // undefined behavior
    auto gamma = std::allocate_shared<L>(NearAllocator<L>()); // OK!
}
```

In my opinion, solutions (1), (2), and (6) are all tenable. Solution (1) is the de-facto solution today: **Near fancy pointer types are just not allowed.**

Scenario C. High-memory-area "far" pointers.

Let *main memory* be defined as the set of all object addresses addressable by C++ native pointer types. Some programmers deal in objects that are not located in main memory.

A historical example is the 80286, where 16-bit native pointers could identify only the memory addresses between 0x0000 and 0xFFFF, but the computer itself had additional memory ranging from address 0x1'0000 to address 0xF'FFFF, and some models had a "high memory area" between 0x10'0000 and 0x10'FFEF. These additional addresses were not addressable with native pointers such as `int*`. Compilers of the day provided additional built-in types such as `far int*`, where `sizeof(far int*) > sizeof(int*)`.

A possible future example is objects residing in some location which *is not memory-mapped into the current process's virtual address space*. Today this is unlikely to be a real problem for 64-bit address spaces — we just take anything we care about (including the OS kernel, and also GPU memory if I understand correctly) and map it into the current process's address space. It might be a problem looking forward into the future of enormous (>18 exabytes?) non-volatile storage, or more likely looking downward into the domain of 32-bit and 16-bit microcontrollers with smaller main memories.

Can we use fancy pointer types to address objects that are not located in main memory?

No, we cannot use fancy pointer types to address objects that aren't located in main memory.

Every fancy pointer type must provide an operator* that returns a reference to an object. That is, we must have a function definition along these lines:

```
struct FancyPtrInt {
    int& FancyPtrInt::operator*() const {
        // some magic goes here
    }
    // ...
};
```

The expression *fancyp returns a reference to its identified object as type int&. So its identified object must be addressable by a native pointer value — namely, std::addressof(*fancyp). Therefore, this attempt to use fancy pointers to address objects outside main memory has failed.

Scenario D. "Fat" pointers carrying metadata for dereference-time.

Some programming systems use "fat pointers" which are *augmented* versions of native pointers. The *metadata* part of a fat pointer communicates the size of the original allocation so that dereferences can be bounds-checked, and/or the type of the original allocation so that dereferences can be type-checked.

For example, here is one possible "fat" fancy pointer type.

```
template<class T>
class FatPtr {
    char *base_ = nullptr;
    int cur_ = 0;
    int max_ = 1;
public:
    FatPtr() = default;
    FatPtr(std::nullptr_t) {}
    explicit FatPtr(T *p, int n) : base_((char*)p), max_(n * sizeof (T)) {}

    template<class U>
    explicit FatPtr(const FatPtr<U>& rhs) :
        base_(rhs.base_), cur_(rhs.cur_), max_(rhs.max_) {
        if (cur_ % sizeof(T) != 0) throw "misaligned";
    }

    T& operator*() const {
        if (base_ == nullptr) {
            throw "null";
        } else if (cur_ < 0 || max_ <= cur_) {
            throw "out of bounds";
        } else if (cur_ % sizeof(T) != 0) {
            throw "misaligned";
        } else {
            return *(T*)(base_ + cur_);
        }
    }

    auto& operator++() {
        if (base_ == nullptr) throw "null";
        if (max_ <= cur_ + sizeof(T)) throw "out of bounds";
        cur_ += sizeof(T);
        return *this;
    }

    // ...
};
```

```

template<class T>
struct FatAllocator {
    using value_type = T;
    using pointer = FatPtr<T>;
    using void_pointer = FatPtr<void>;

    pointer allocate(size_t n) {
        T *p = std::allocator<T>().allocate(n);
        return pointer(p, n);
    }
    void deallocate(pointer p, size_t n) {
        if (p.cur_ != 0) throw "deallocating non-allocated pointer";
        std::allocator<T>().deallocate(p, n);
    }
};

```

This fat-pointer type plays well with list-like containers because its range of values includes all of main memory.

In order to get the benefit of bounds-checking, the container must do pointer arithmetic only on `FatPtr` values losslessly derived from the originally allocated `FatPtr`.

In order to get the benefit of *type*-checking, the container must dereference only `FatPtr` values losslessly derived from the originally allocated `FatPtr`, which again implies doing pointer arithmetic only on `FatPtr`. This will generally have a performance cost, especially in debug mode. Worse, if we use fat pointers *only* for safety-checking, there will be no benefit to counter the performance cost, unless the standard container itself has bugs.

The fat-pointer scenario is the only scenario in this paper where it makes sense to do pointer arithmetic on fancy pointers. In every other case, it is equivalent — and generally more efficient, especially in debug mode — to do pointer arithmetic only on native pointers.

Can we use *fat fancy pointer* types to bounds-check each access to an allocation?

Yes; but I think it is not worth the performance cost.

We have the following possible solutions to this problem:

1. Fat fancy pointer types are technically permitted, but may be sliced to native pointers at any time. Every fancy pointer type is required to model `RandomAccessIterator`; containers may use either fancy pointers or native pointers for pointer arithmetic. This is the de-facto and most conservative solution.
2. Fat fancy pointer types are fully supported. Every fancy pointer type is required to model `RandomAccessIterator`. Containers must do pointer arithmetic only on fancy pointers derived from the original allocation. This solution has a performance cost in debug mode. This solution also has the cost that it transforms the *de-jure* requirement (that fancy pointer types model `RandomAccessIterator`) into a *de-facto* requirement. User-defined fancy pointer types that do not model all of `RandomAccessIterator` may stop working if this solution is implemented. This solution requires vendors to do a lot of new work.
3. Fat fancy pointer types are supported; but containers are required to use them for pointer-arithmetic only in the implementations of functions which might have undesirable behavior if done with native pointers. For example, the bounds-checked `vector::at` could use either native or fancy pointer arithmetic, but the unchecked `vector::operator[]` must dereference a fancy pointer losslessly derived from the original allocation. The Standard would have to specify "fancy" or "native" arithmetic for each container member function. This solution requires vendors to do some new work.
4. Fat fancy pointers are explicitly unsupported. Fancy pointers are no longer required to model `RandomAccessIterator` at all. Containers must slice fancy pointers to native pointers before performing any operation other than comparison, dereference, or deallocation. This solution requires vendors to do some new work.

In my opinion, solutions (1), (3), and (4) are tenable. Solution (2) has too many practical disadvantages: breaks working code, hurts performance, requires work from vendors. I claim that solution (4) is the best: **Fat pointers should remain unsupported. Fancy pointer types should no longer be required to model `RandomAccessIterator`.**

Scenario E. "Segmented" pointers carrying metadata for deallocate-time.

Some programming systems use *augmented* pointers similar to Scenario D, but whose *metadata* is used only by the deallocation mechanism. For example, a memory resource managing several memory "segments" might store the identity of the current segment in the allocated pointer, so that the pointer can be returned to the correct segment at deallocation time. Because of this example, I call such fancy pointers *segmented pointers*.

For example, here is one possible "segmented" fancy pointer type. [A full implementation is available on my GitHub under the name `scratch::pmr::segmented\_resource`.](#)

```
class Segment;

template<class T>
class SegmentedPtr {
    T *ptr_ = nullptr;
    Segment *seg_ = nullptr;
public:
    SegmentedPtr() = default;
    SegmentedPtr(std::nullptr_t) {}
    explicit SegmentedPtr(T *p, Segment *seg) : ptr_(p), seg_(seg) {}

    template<class U>
    explicit SegmentedPtr(const SegmentedPtr<U>& rhs) :
        ptr_(rhs.ptr_), seg_(rhs.seg_) {}

    auto segment() const { return seg_; }
    T& operator*() const { return *ptr_; }
    auto& operator++() { ++ptr_; return *this; }

    // ...

    static auto pointer_to(T& r) { return SegmentedPtr(&r, nullptr); }
};

class Segment {
    char buffer[10000];
    int index = 0;
    int freed = 0;
public:
    bool can_allocate(size_t bytes) {
        return (sizeof buffer - index) >= bytes;
    }
    auto allocate(size_t bytes) {
        index += bytes;
        void *p = &buffer[index - bytes];
        return SegmentedPtr<void>(p, this);
    }
    void deallocate(void *, size_t bytes) {
        freed += bytes;
        if (freed == index) {
            index = freed = 0;
        }
    }
};

class SegmentedResource {
    std::list<Segment> m_segments;
public:
    SegmentedPtr<void> allocate(size_t bytes, size_t align) {
        assert(align <= alignof(std::max_align_t));
        bytes += -bytes % alignof(std::max_align_t);
        assert(bytes <= 10000);

        for (auto&& seg : m_segments) {
            if (seg.can_allocate(bytes)) {
                return seg.allocate(bytes);
            }
        }
        return m_segments.emplace_back().allocate(bytes);
    }
    void deallocate(SegmentedPtr<void> p, size_t bytes, size_t) {
        bytes += -bytes % alignof(std::max_align_t);
        p.segment()->deallocate(static_cast<void*>(p), bytes);
    }
};

template<class T>
class SegmentedAllocator {
    SegmentedResource *mr_;
public:
    using value_type = T;
    using pointer = SegmentedPtr<T>;

    SegmentedAllocator(SegmentedResource *mr) : mr_(mr) {}
};
```

```

    pointer allocate(size_t n) {
        return pointer(mr_>allocate(n * sizeof(T), alignof(T)));
    }
    void deallocate(pointer p, size_t n) {
        return mr_>deallocate(p, n * sizeof(T), alignof(T));
    }
};

int main() {
    SegmentedResource mr;
    std::list<int, SegmentedAllocator<int>> widgets(&mr);

    widgets.push_back(1);
}

```

The important line is this one, in `SegmentedResource::deallocate`:

```
p.segment()->deallocate(static_cast<void*>(p), bytes);
```

This line returns `p`'s memory to the segment from which it was originally allocated. If the identity of that segment were not part of `p`'s metadata, this code would not work. If `p`'s metadata had been sliced away prior to deallocation (that is, if the pointer passed to the deallocation mechanism were not losslessly derived from the original allocation), this code would not work.

If the programmer cannot rely on the container to deallocate a pointer losslessly derived from the original allocation, then the programmer cannot use this kind of segmented memory resource.

Can we use *segmented pointer* types to hold metadata used at deallocation time?

Yes, we can; it has practical difficulties, but I believe it is worth implementing.

The problems in this case are not with the containers. No vendor's containers actually support segmented pointers today, due to needless overuse of slicing expressions such as `std::pointer_traits<P>::pointer_to(std::addressof(*q))`; but adjusting those expressions to non-slicing expressions such as `static_cast<P>(q)` can be done mechanically. Some vendors' containers (e.g. `libc++`) already use fancy pointer types internally (in order to support Scenario A), so no ABI-breaking changes to class layouts would be required in order to support segmented pointers. Other vendors (e.g. `libstdc++`) use native pointer types internally, and thus even to support Scenario A would involve ABI-breaking changes for them.

Even `libc++` has ABI problems with the allocator-aware non-containers: `promise`, `shared_ptr`, and `packaged_task`. These types have shared states that may be allocated with an allocator. Their current implementations generally do not preserve a copy of the allocated *pointer* — only a copy of the allocator itself.

For example, `libc++`'s `promise(allocator_arg, _Alloc)` looks effectively like this:

```

template<class _Rp, class _Alloc>
class __assoc_state_alloc : public __assoc_state<_Rp>
{
    _Alloc __alloc_;
    void __on_zero_shared() noexcept override {
        using _Al = typename __allocator_traits_rebind<_Alloc, __assoc_state_alloc>::type;
        using _ATraits = allocator_traits<_Al>;
        using _PTraits = pointer_traits<typename _ATraits::pointer>;
        _Al __a(__alloc_);
        this->~__assoc_state_alloc();
        __a.deallocate(std::pointer_traits<_P>::pointer_to(*this), 1);
    }
public:
    explicit __assoc_state_alloc(const _Alloc& __a)
        : __alloc_(__a) {}
};

template<class _Rp>
template<class _Alloc>
promise<_Rp>::promise(allocator_arg_t, const _Alloc& __a0)
{
    using _State = __assoc_state_alloc<_Rp, _Alloc>;
    using _A2 = typename __allocator_traits_rebind<_Alloc, _State>::type;
    using _D2 = __allocator_destructor<_A2>;
    _A2 __a(__a0);
    unique_ptr<_State, _D2> __hold(__a.allocate(1), _D2(__a, 1));
    ::new(static_cast<void*>(std::addressof(*__hold.get()))) _State(__a0);
}

```

```

    __state_ = std::addressof(*__hold.release());
}

```

The argument to `__a.deallocate` is the direct result of `pointer_to`; that is, it will fail to correctly reconstitute the metadata that was lost in the conversion from fancy pointer `__hold.get()` to native pointer `this`. To make this code work with segmented pointers, the `__assoc_state_alloc` struct must store a copy of the originally allocated pointer.

We have the following possible solutions to this problem:

1. Segmented pointer types are not allowed. Fancy pointers may be sliced to native pointers at any time, which means their metadata cannot be used for deallocation. This is the de-facto solution.
2. Segmented pointer types are fully supported. Every allocator-aware type is required to use for deallocation a fancy pointer derived from the corresponding original allocation. This solution requires vendors to do a lot of new work.
3. Segmented pointer types are supported by some allocator-aware types, and not supported by others. The Standard dictates which allocator-aware types are *segmented-allocator-aware*. This solution requires specification work from the Committee, plus some work from vendors, and leaves the situation confusing for users. (Therefore this solution is summarily dismissed.)

In my opinion, solutions (1) and (2) are tenable.

Is any kind of fancy pointer compatible with C++?

We have seen that there are five plausible scenarios for fancy-pointer usage in C++.

Scenario (A), offset pointers, has only one tenable solution, "continue to partly support."

Scenario (B), near pointers, has three possible solutions of which the de-facto one is "do not support."

Scenario (C), far pointers, was proven impossible to support.

Scenario (D), fat pointers, has three possible solutions of which the de-facto one is "do not support."

Scenario (E), segmented pointers, has two possible solutions of which the de-facto one is "do not support."

If we adopt the de-facto solution in all scenarios, our conclusion is that only Scenario (A) is tenable, and therefore we may want to explore ways to separate *storage type* from *allocation mechanism*; we also may want to explore ways to determine the *representation-awareness* of a type at compile time.

If we are willing to force some work on vendors, my preferred solutions would be A-1 (continue the status quo on offset pointers), B-6 (support near pointers, with undefined behavior for certain uses of list-like containers); D-1 (do not support fat pointers), E-2 (fully support segmented pointers). I will refer to this set of solutions as *fancy-pointer-support*.

Requirements on fancy pointer types if *fancy-pointer-support* were adopted.

Solution B-6 requires only that it be possible to cast a native pointer-to-Base (to a subobject of a list-like container; therefore, non-null) into a *dereferenceable* fancy pointer-to-Base. This requirement is translated into C++ terms via the standard trait

```
std::pointer_traits<FancyPtrT>::pointer_to(T& r).
```

The reason we must use `std::pointer_traits<FancyPtrT>::pointer_to(T& r)` rather than the simpler `static_cast<FancyPtrT>(T*)` is that there may not be any natural mapping from `T*` values onto `FancyPtrT` values. In scenario (B), some values of `T*` do not have an image in `FancyPtrT`. In scenario (E), the operation requires us to invent the metadata part of a `FancyPtrT` value. In both cases a mapping is possible, but it is not a *natural* mapping, and thus must not be represented in C++ by `static_cast`.

It is a shame that the signature of `pointer_to` takes `T&`, thus requiring that the input native pointer be *dereferenceable*. This means there is no generic way to get the fancy-pointer value corresponding to a one-past-the-end pointer; and the generic way to get a fancy null pointer is `FancyPtrT{}` rather than `pointer_to(nullptr)`. This does not seem to be a problem for containers in practice.

If scenario (A), offset pointers, is the only scenario in play, then `static_cast<FancyPtrT>(T*)` is perhaps an appropriate way to get the storage representation of a native pointer. Scenario (B), near pointers, is also relevant to storage representation.

Solution E-2 requires that it be possible to losslessly cast a fancy pointer-to-Derived into a fancy-pointer-to-Base, and also vice versa. (Here Base represents a possibly-terminal node of a list-like container, and Derived represents a non-terminal node.) The most natural way to express casting in C++ is via `static_cast`.

As long as fancy pointer types are permitted by the Standard, container implementations will also require that it be possible to cast a fancy pointer-to-T into a native pointer-to-T. The most natural way to express casting in C++ is via `static_cast`. Therefore we should require that every fancy pointer type support casting to its corresponding native pointer type.

Today, container implementations say `std::addressof(*fancyPtrT)` instead of `static_cast<T*>(fancyPtrT)`. The latter is more philosophically appropriate. The former also has the potential problem that it can be used only for *dereferenceable* pointer values. `std::addressof(*fancyPtrT)` cannot safely be used when `fancyPtrT == nullptr`. This does not seem to be a problem for containers in practice.

Also, `std::addressof(*fancyPtrT)` cannot safely be used when `fancyPtrT` represents a "one-past-the-end" fancy pointer value; but such values arise only via fancy pointer arithmetic.

If we forbid *fat pointers*, then no container implementation will ever require fancy pointer arithmetic. Therefore we should drop the requirement that fancy pointer types model `RandomAccessIterator`. This may require some work from vendors, to replace expressions such as `fancyPtrT[k]` with `static_cast<T*>(fancyPtrT)[k]`.

In my preferred solution (support A, B, and E but not D), every STL container needs to follow these rules:

1. Any dereferenceable pointer stored "within" the container must be stored as a fancy pointer. (Required by A; wanted by B.)
2. A dereferenceable pointer stored "outside" the container, such as in an iterator, may be stored as a fancy pointer or as a native pointer. (Relevant to A.)
3. Whenever you get a pointer from `allocate()`, hang onto it. You will never be able to reconstruct its metadata if you lose it. (Required by E.)
4. Assume that metadata is preserved via the fancy pointer type's special member functions, and via casting from one fancy pointer type to a rebound version of that type (e.g. `static_cast<FancyPtrT>(fancyPtrU)`). (This proposes a resolution to LWG 2260.) Do not assume that metadata is preserved via any other operations. (Required by E.)
5. Assume that `static_cast<T*>(fancyPtrT)` is a correct slicing expression. Prefer to avoid `std::addressof(*fancyPtrT)`, which isn't going to work if `fancyPtrT` is not a *dereferenceable* value. (Required by A, B, and E.)
6. Assume that `std::pointer_traits<FancyPtrT>::pointer_to(t)` returns a *dereferenceable* pointer, but not a *deallocatable* one. Do not assume that `static_cast<FancyPtrT>(std::addressof(t))` compiles; `FancyPtrT` might not have a one-argument constructor. (Required by E.)
7. Assume that the fancy pointer type models `NullablePointer`. (Construction from and comparison to `nullptr` ought to be supported.)
8. Do not assume that the fancy pointer type models `RandomAccessIterator`. For example, do not rely on `operator++`, `operator[]`, or `operator<`. Slice to `T*` before attempting these operations. (Improves the performance and usability of A, B, and E. Precludes D.)

basic_string requires trivial default constructibility.

The `basic_string` implementations of `libc++` and `MSVC` implicitly require that their allocator's pointer type be trivially default constructible and trivially destructible ([libc++ bug 20508](#)). The Standard should either explicitly specify these requirements in `[string.require]`, or else the implementations of `basic_string` should be fixed to avoid implicitly depending on trivial constructibility and destructibility of the pointer type.

Appendix. Representation-awareness of container and iterator types, in some present-day library implementations.

The following table shows, for various container classes `C` across various common library implementations, the compatibility of `C` with `boost::interprocess::offset_ptr`. This demonstrates the current state of my Scenario A (*offset pointers*) across these implementations.

	libc++	GNU	MSVC	Boost.Container
<code>deque</code>	Yes	Yes	Yes	Yes
<code>forward_list</code> (slist)	Yes	No	Yes	Yes

list	Yes	No	Yes	Yes
map (etc.)	Yes	No	Yes	Yes
unordered_map (etc.)	Yes	No	Yes	
flat_map (etc.)				Yes
vector	Yes	Yes	Yes	Yes
basic_string	X	Yes	X	Yes

The "X" entries for `basic_string` indicate that the libraries require trivial constructibility, which `offset_ptr` does not have; but if the library vendors patched this one bug, then the libraries would be able to support `offset_ptr` with no further patches (as far as I know).

The following table shows, for various container classes `C` across various common library implementations, the compatibility of `C::iterator` with `boost::interprocess::offset_ptr`. This demonstrates the current state of my Scenario A (*offset pointers*) across these implementations.

	libc++	GNU	MSVC	Boost.Container
deque iterator	Yes	Yes	No	Yes
forward_list (slist) iterator	Yes	No	Yes	Yes
list iterator	Yes	No	Yes	Yes
map (etc.) iterator	Yes	No	Yes	Yes
unordered_map (etc.) iterator	Yes	No	Yes	
flat_map (etc.) iterator				Yes
vector iterator	Yes	Yes	Yes	Yes
basic_string iterator	Yes	Yes	Yes	Yes

GNU `forward_list`, `list`, `map`, and `unordered_map` iterators each hold a raw pointer to an allocated list node. MSVC `deque`'s iterator holds a raw pointer to the deque itself, and an integer offset.

Appendix. References and further reading.

Boost.Interprocess `offset_ptr` documentation.

https://www.boost.org/doc/libs/1_65_0/doc/html/interprocess/offset_ptr.html

Lance Diduck. N2486 "Alternative Allocators and Standard Containers." December 2007. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2486.pdf>

Nevin Liber. N3884 "Contiguous Iterators: A Refinement of Random-Access Iterators." January 2014.

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n3884.pdf>

Doug Judd. "Why `pointer_traits` was introduced in C++11." September 2015.

http://blog.nuggetwheat.org/index.php/2015/09/01/why-pointer_traits-was-introduced-in-c11/

Thomas Köppe. "A visitor's guide to C++ allocators." September 2016. https://rawgit.com/google/cxx-std-draft/allocator-paper/allocator_user_guide.html

Arthur O'Dwyer. "pointer-traits.md." August 2017. <https://github.com/Quuxplusone/from-scratch/blob/master/include/scratch/bits/traits-classes/pointer-traits.md>

Jonathan Wakely. LWG 2260 "Missing requirement for `Allocator::pointer`." Opened 2013-05-14, last modified 2017-07-30. <https://timsong-cpp.github.io/lwg-issues/2260>

The most current revision of P0773 "Towards Meaningful Fancy Pointers" is available at <https://quuxplusone.github.io/draft/fancy-pointers.html>

Acknowledgments.

Thanks to Alfredo Correa for identifying many confusing passages in drafts of this paper.